C950 Task-2 WGUPS Write-Up

(Task-2: The implementation phase of the WGUPS Routing Program).

(Zip your source code and upload it with this file)

Kalvin Patarawong

Student ID:

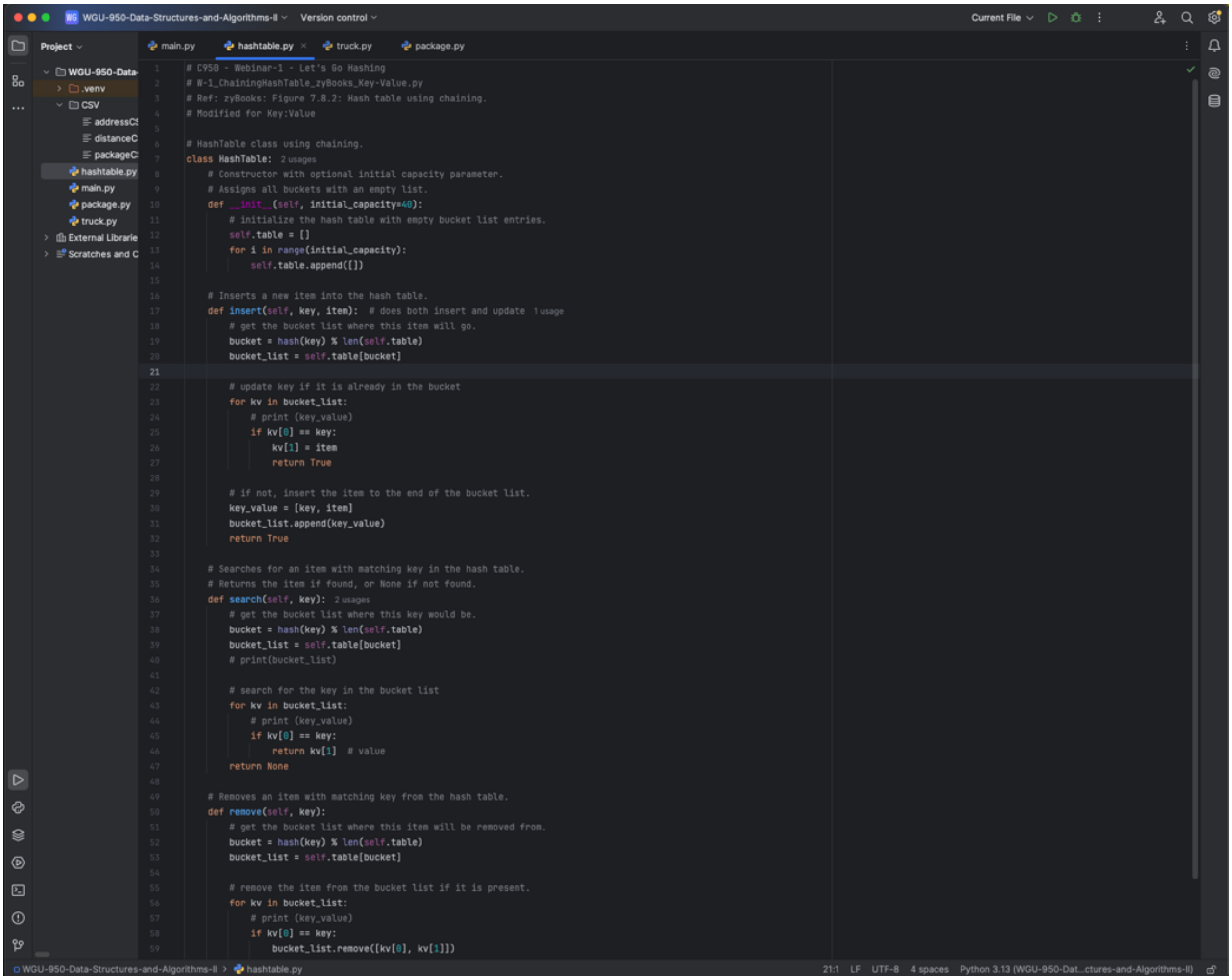
WGU Email:

Date: 10/7/2024

C950 Data Structures and Algorithms II

A. Hash Table

Hash Table Screenshot goes here



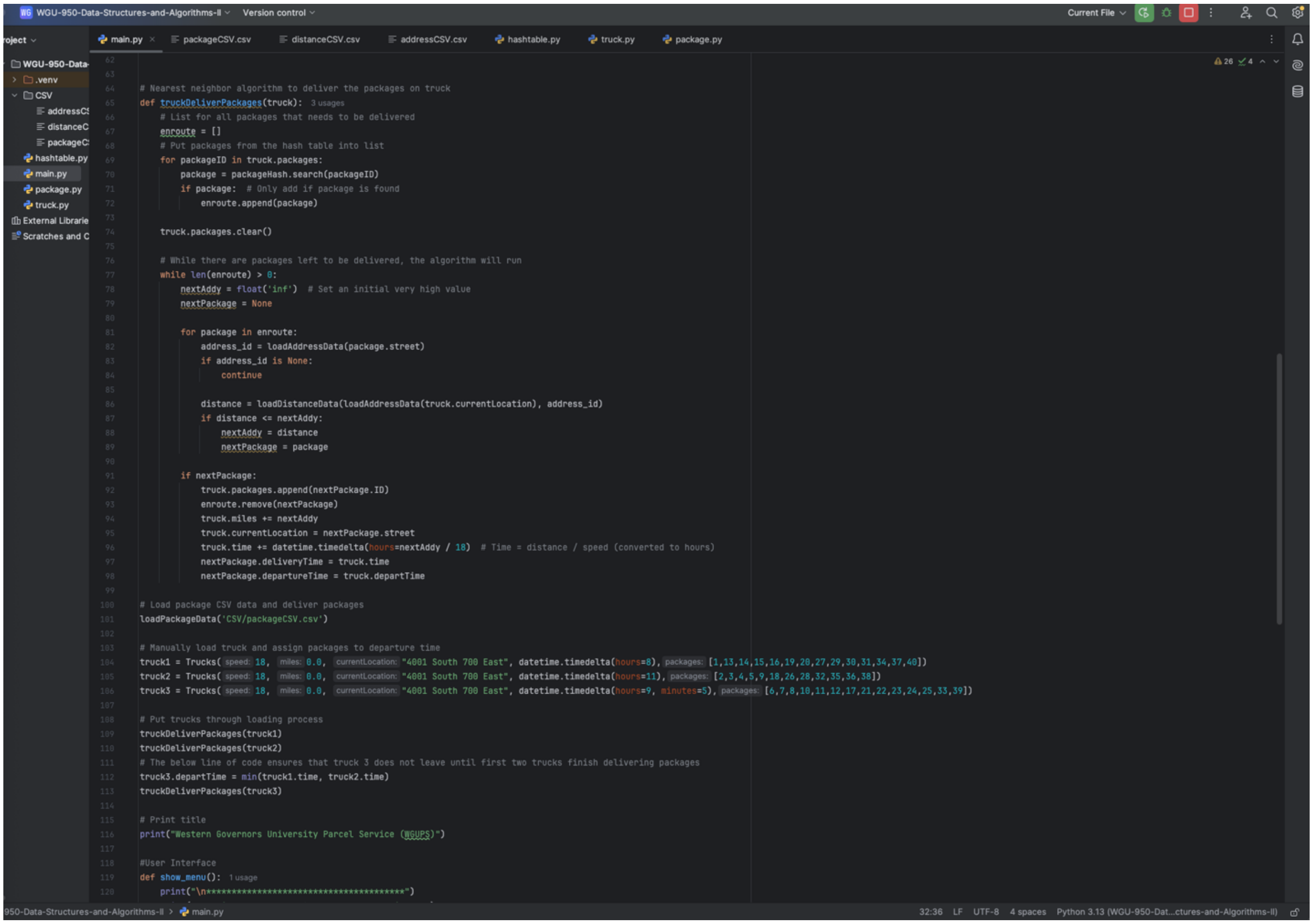
B. Look-Up Functions

Look-up function screenshot goes here

```python

# Load distance data between two locations
def loadDistanceData(x_value, y_value):  1 usage
    distance = DistanceCSV[x_value][y_value]
    if distance == '':
        distance = DistanceCSV[y_value][x_value]
    return float(distance)

# Load address data for matching delivery location
def loadAddressData(address):  2 usages
    for row in AddressCSV:
        if address in row[2]:
            return int(row[0])
    return None

```

C. Original Code

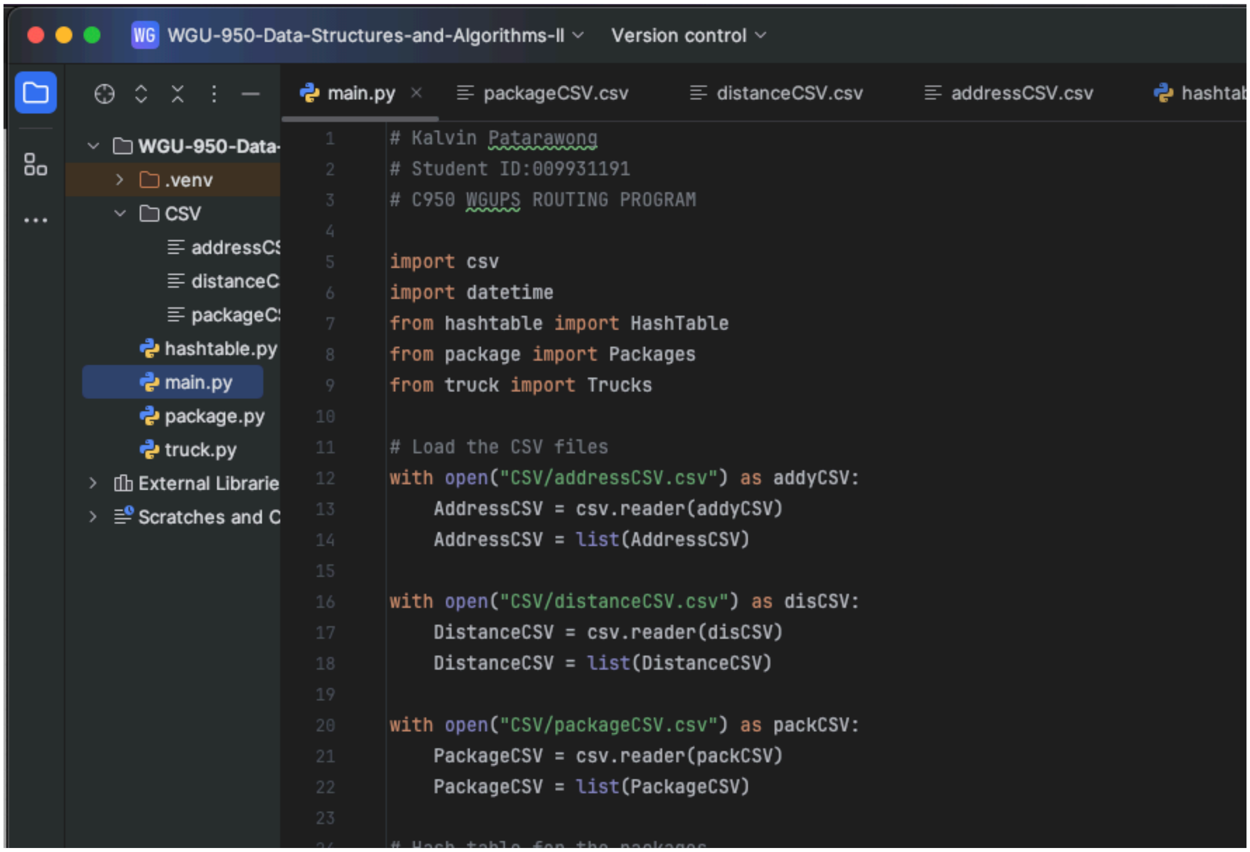
Major code blocks screenshots go here showing implementation



```python
# Nearest neighbor algorithm to deliver the packages on truck
def truckDeliverPackages(truck):
    # List for all packages that needs to be delivered
    enroute = []
    # Put packages from the hash table into list
    for packageID in truck.packages:
        package = packageHash.search(packageID)
        if package:  # Only add if package is found
            enroute.append(package)

    truck.packages.clear()

    # While there are packages left to be delivered, the algorithm will run
    while len(enroute) > 0:
        nextAddy = float('inf')  # Set an initial very high value
        nextPackage = None

        for package in enroute:
            address_id = loadAddressData(package.street)
            if address_id is None:
                continue

            distance = loadDistanceData(loadAddressData(truck.currentLocation), address_id)
            if distance <= nextAddy:
                nextAddy = distance
                nextPackage = package

        if nextPackage:
            truck.packages.append(nextPackage.ID)
            enroute.remove(nextPackage)
            truck.miles += nextAddy
            truck.currentLocation = nextPackage.street
            truck.time += datetime.timedelta(hours=nextAddy / 18)  # Time = distance / speed (converted to hours)
            nextPackage.deliveryTime = truck.time
            nextPackage.departureTime = truck.departTime

# Load package CSV data and deliver packages
loadPackageData('CSV/packageCSV.csv')

# Manually load truck and assign packages to departure time
truck1 = Trucks(18, 0.0, "4001 South 700 East", datetime.timedelta(hours=8), [1,13,14,15,16,19,20,27,29,30,31,34,37,40])
truck2 = Trucks(18, 0.0, "4001 South 700 East", datetime.timedelta(hours=11), [2,3,4,5,9,18,26,28,32,35,36,38])
truck3 = Trucks(18, 0.0, "4001 South 700 East", datetime.timedelta(hours=9, minutes=5), [6,7,8,10,11,12,17,21,22,23,24,25,33,39])

# Put trucks through loading process
truckDeliverPackages(truck1)
truckDeliverPackages(truck2)
# The below line of code ensures that truck 3 does not leave until first two trucks finish delivering packages
truck3.departTime = min(truck1.time, truck2.time)
truckDeliverPackages(truck3)

# Print title
print("Western Governors University Parcel Service (WGUPS)")

#User Interface
def show_menu():
    print("\n**************************************")
```

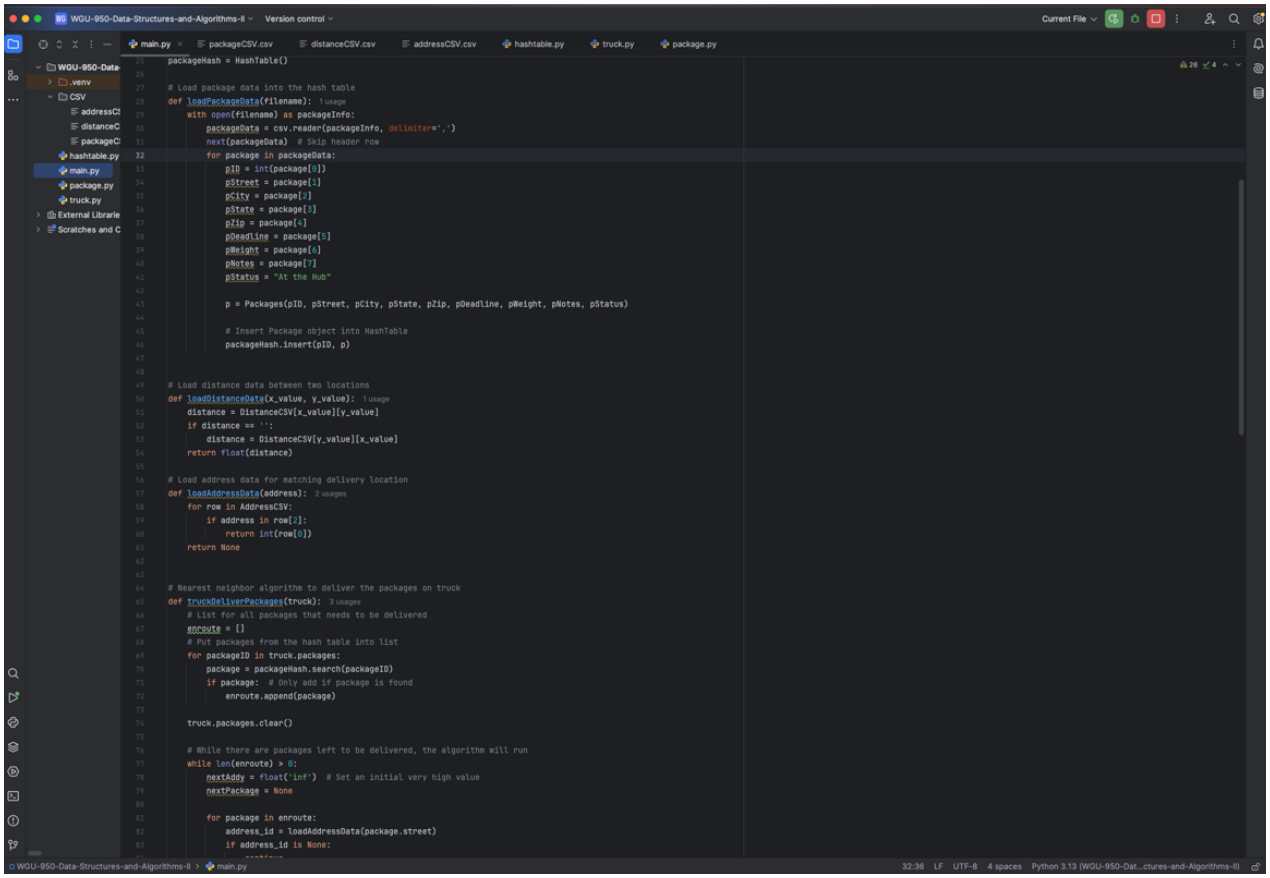
C1. Identification Information

main.py screenshot goes here showing Student ID



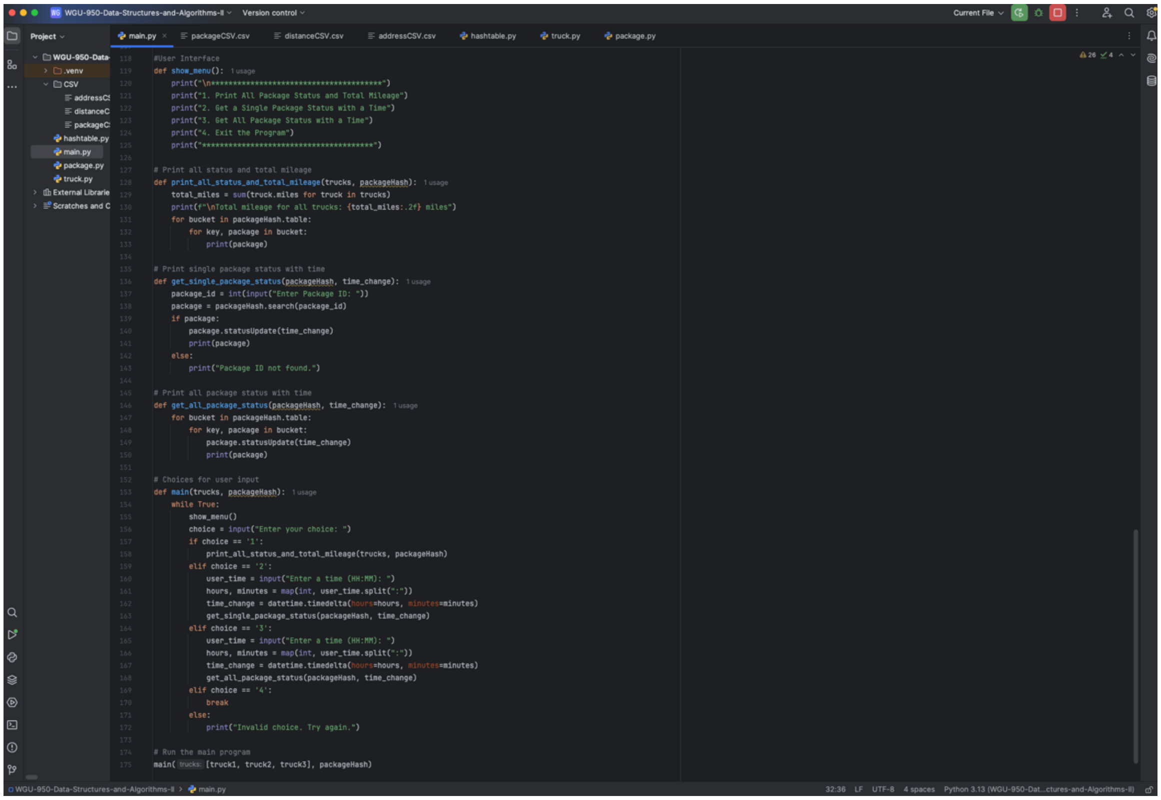
C2. Process and Flow Comments

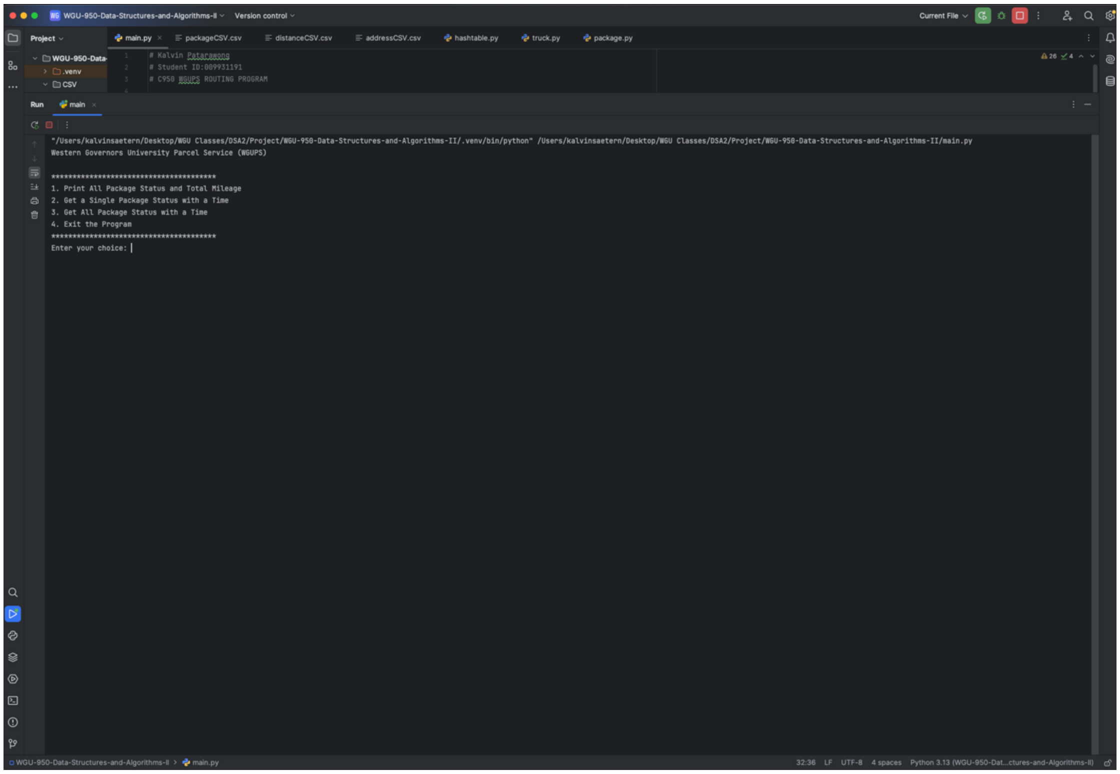
Some code blocks screenshots go here showing comments



D. Interface

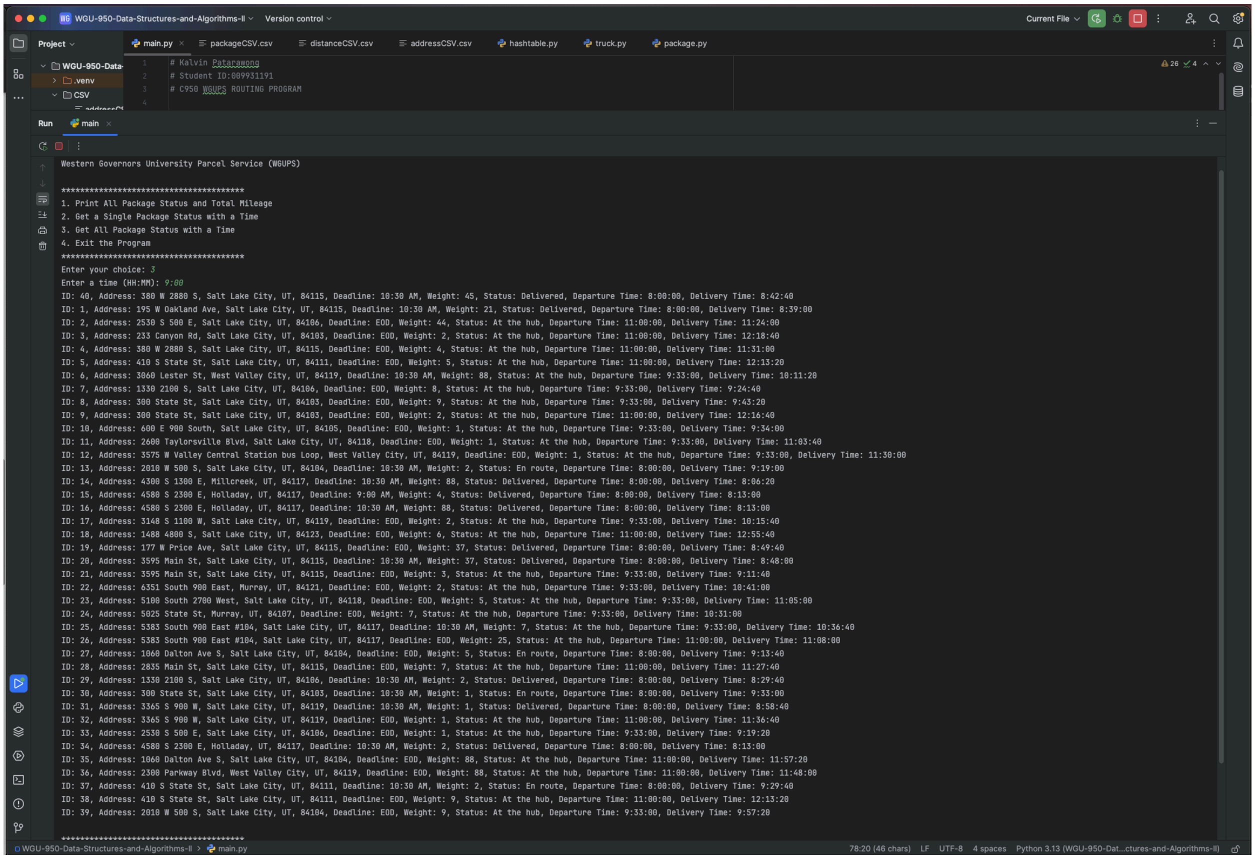
Interface screenshot goes here





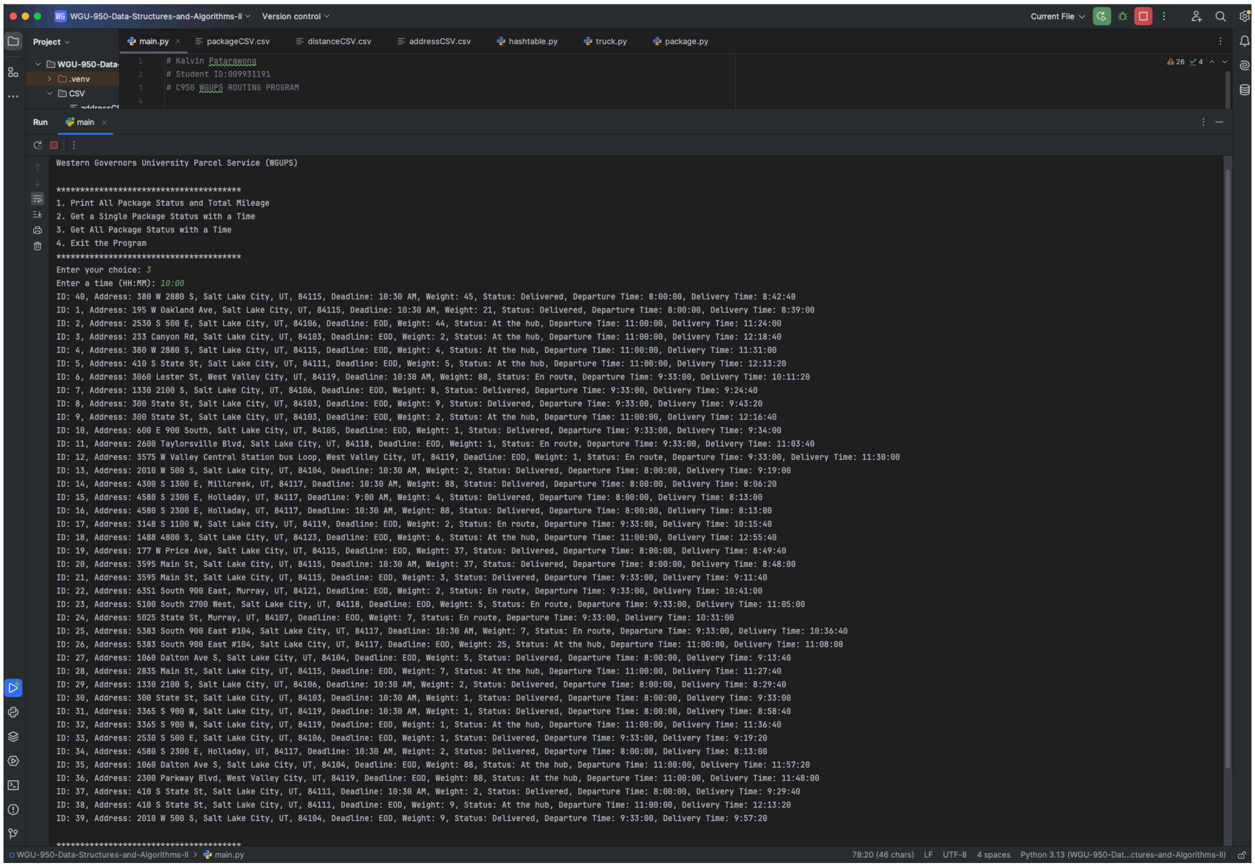
D1. First Status Check

Screenshot goes here



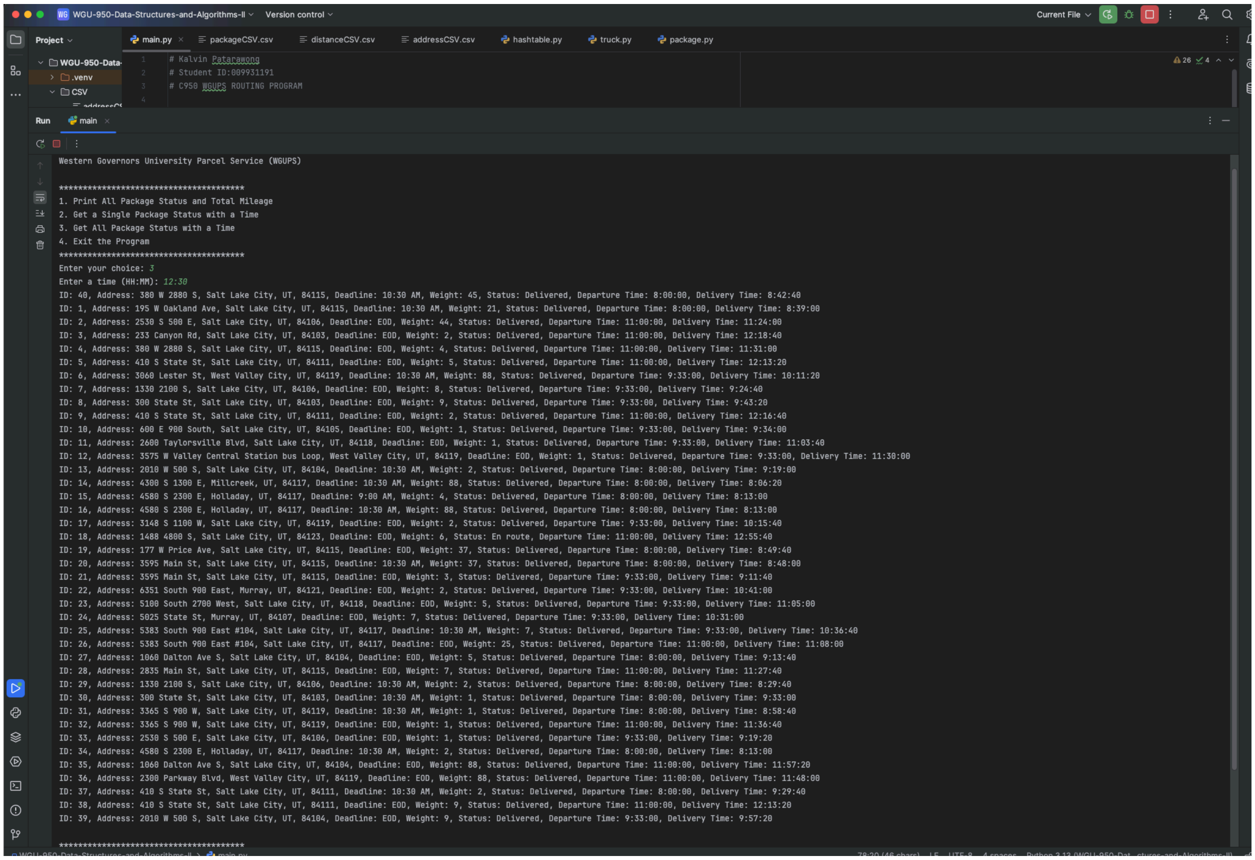
D2. Second Status Check

Screenshot goes here



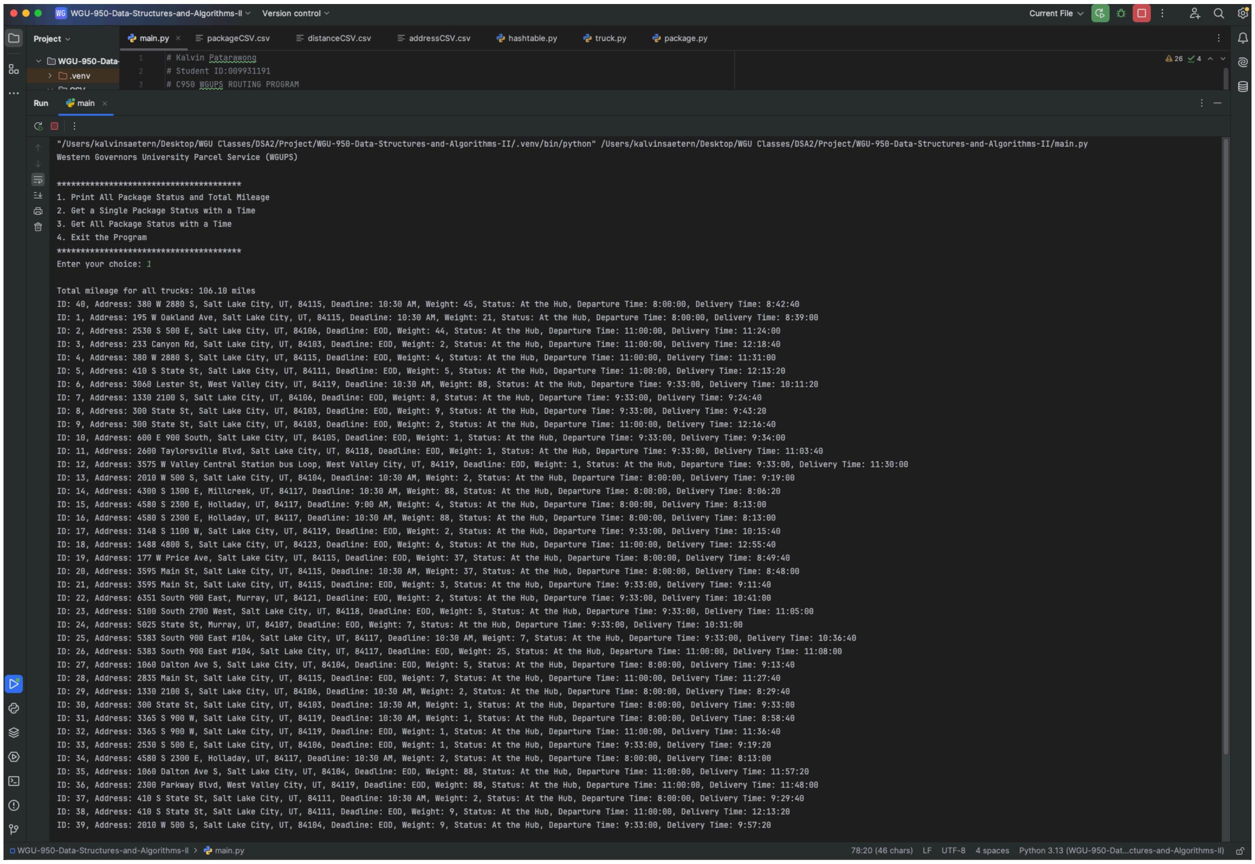
D3. Third Status Check

Screenshot goes here



E. Screenshot of Code Execution

Screenshot goes here



F1. Strengths of the Chosen Algorithm

The package delivery algorithm used in the solution, based on the Nearest Neighbor Algorithm, has multiple strengths. It is efficient, offering a quick solution to route planning and reducing complexity. It is also simple and easy to understand and maintain, making it ideal for smaller-scale scenarios. The algorithm can handle multiple classes and works well with non-linear data, ensuring reliability even when the path between delivery points is unclear. These strengths make it a practical choice for the delivery problem in the original program.

F2. Verification of Algorithm

The Nearest Neighbor Algorithm effectively meets all the requirements for this scenario by ensuring all packages are delivered on time, keeping the total mileage under 140 miles, and accommodating special delivery instructions. Packages that couldn't depart until 9:05 were scheduled accordingly, and those that needed to be delivered together were handled on the same trip. The algorithm guarantees that no package is left behind, fulfilling the delivery process's time and logistical needs.

F3. Other Possible Algorithms

Two other algorithms that could meet the requirements in this scenario are Dynamic Programming and Genetic Algorithm. Dynamic Programming could solve the Traveling Salesman Problem with time windows, ensuring an optimal solution, though with higher computational complexity. Genetic Algorithms, on the other hand, could optimize routes by evolving solutions over multiple generations, considering constraints like time windows, deadlines, and total mileage.

F3a. Algorithm Differences

Dynamic Programming differs from the algorithm used in the solution because it guarantees an optimal route but comes with a higher computational cost. DP evaluates all possible routes and eliminates non-optimal ones, which can take a significant amount of time.

On the other hand, the Genetic Algorithm uses a probabilistic approach to evolve solutions over multiple generations. While it doesn't always guarantee the optimal solution, it can find a good one more quickly, making it a more efficient option in terms of computational time.

G. Different Approach

If I were to do this project again, I'd make a few changes to improve it. First, I'd add a feature to prioritize packages based on their deadlines or special instructions, ensuring those are delivered first. I'd also refactor the code to sort packages more efficiently, making the delivery process smoother. Finally, I'd work on making the user interface more interactive and user-friendly so it's easier to navigate and use.

H. Verification of Data Structure

The hash table used in the solution meets the scenario's requirements by efficiently storing and accessing package data. With each package having a unique ID, the hash table allows for quick lookups, which is vital for retrieving package details during delivery. This speed is essential for the algorithm to work efficiently. The hash table also offers flexibility-packages can be added or removed as needed when trucks are loaded, and their details can be easily updated. This dynamic functionality ensures that the program can adapt to changes in the delivery process while maintaining accuracy and performance.

H1. Other Data Structures

Two other data structures that could meet the requirements are a doubly linked list, which allows efficient insertion and deletion of packages with bidirectional traversal, and a queue, which works well for managing packages in a first-in-first-out order for sequential processing.

H1a. Data Structure Differences

The doubly linked list differs from the hash table because it stores elements in a sequence with pointers to the previous and next nodes, making inserting and deleting packages efficient. However, searching for a specific package takes longer, with an $O(n)$ time complexity. In contrast, a hash table allows for constant-time lookups $O(1)$ by using the unique package ID as the key, making it much faster for searching and retrieving data.

On the other hand, the queue processes data in a first-in-first-out (FIFO) order, which works well for sequentially handling packages. However, this structure isn't as flexible when dealing with packages that need to be accessed or delivered out of sequence, as it processes them in a strict order. The hash table, by contrast, gives direct access to packages based on their ID, making it better suited for handling more complex delivery requirements like deadlines or specific routes.

I. Sources

Lysecky, R., & Vahid, F. (2018, June). *C950: Data Structures and Algorithms II*. zyBooks.

Retrieved March 22, 2021, from https://learn.zybooks.com/zybook/WGUC950AY20182019/

C950 WGUPS Project Implementation Steps - Example - Nearest Neighbor. (n.d.).

https://srm-c.vf.force.com/apex/coursearticle?Id=kA03x000001DbBGCA0